CS 374: Section A - Homework 7, Problem 1

Anirudh Konidala

October 20, 2026

(a) Longest common decreasing subsequence

Throughout, "decreasing" means strictly decreasing: every element is smaller than the element before it. The empty sequence and every one-element sequence are decreasing, so the answer is always at least 0.

Recursive functions

We use two functions. For $0 \le i \le m$ and $1 \le j \le n$, define:

> $LCDS(i, j)$ = the length of the longest common subsequence of the prefixes $A[1..i]$ and $B[1..j]$ that is decreasing and ends with the element $B[j]$, or 0 if no such subsequence exists.

For $1 \le i \le m$ and $0 \le j \le n$, define:

> $Above(i, j)$ = the largest value $LCDS(i-1, j')$ over all indices $1 \le j' \le j$ with $B[j'] > A[i]$, or 0 if no such index j' exists.

In words: $Above(i, j)$ is the length of the longest common decreasing subsequence of $A[1..i-1]$ and $B[1..j]$ whose last element is larger than $A[i]$. Such a subsequence can be extended by the value $A[i]$ and stay decreasing.

Every nonempty common decreasing subsequence of A and B ends with some element $B[j]$, so we need:

$$\max\Big\{0, \max_{1 \le j \le n} LCDS(m, j)\Big\}.$$

Recurrence

$$LCDS(i,j) = \begin{cases} 0 & \text{if } i = 0 \\ LCDS(i-1, j) & \text{if } i \ge 1 \text{ and } A[i] \ne B[j] \\ \max\{LCDS(i-1, j), 1 + Above(i, j-1)\} & \text{if } i \ge 1 \text{ and } A[i] = B[j] \end{cases}$$

$$Above(i,j) = \begin{cases} 0 & \text{if } j = 0 \\ \max\{Above(i, j-1), LCDS(i-1, j)\} & \text{if } j \ge 1 \text{ and } B[j] > A[i] \\ Above(i, j-1) & \text{if } j \ge 1 \text{ and } B[j] \le A[i] \end{cases}$$

Base cases: If $i = 0$, the prefix $A[1..0]$ is empty, so no subsequence ends with $B[j]$ and $LCDS(0, j) = 0$. If $j = 0$, there is no index j', so $Above(i, 0) = 0$.

Recursive case for *LCDS*: Let Z be a longest common decreasing subsequence of $A[1..i]$ and $B[1..j]$ that ends with $B[j]$. Fix one way to embed Z into $A[1..i]$. There are two cases:

- ***Z* does not use *A[i]*:** Then Z is a common decreasing subsequence of $A[1..i-1]$ and $B[1..j]$ that ends with $B[j]$, so its length is at most $LCDS(i-1, j)$. This case is always possible, so $LCDS(i-1, j)$ is always one option.
- ***Z* uses *A[i]*:** The last element of Z is matched to both $A[i]$ and $B[j]$, so this case needs $A[i] = B[j]$. Remove the last element from Z and call the rest Z'. If Z' is empty, $|Z| = 1 \le 1 + Above(i, j-1)$. Otherwise Z' is a common decreasing subsequence of $A[1..i-1]$ and $B[1..j-1]$, and it ends with some $B[j']$ with $j' < j$ and $B[j'] > B[j] = A[i]$. So $|Z'| \le LCDS(i-1, j') \le Above(i, j-1)$. Conversely, the subsequence that gives $Above(i, j-1)$ ends with a value larger than $A[i]$, so we can append $A[i] = B[j]$ to it. This gives a common decreasing subsequence of length $1 + Above(i, j-1)$ that ends with $B[j]$.

Recursive case for *Above*: The indices $j' \le j$ are the indices $j' \le j-1$ plus the index j. The index j counts only if $B[j] > A[i]$, and then it contributes $LCDS(i-1, j)$.

Memoization and evaluation order

We store $LCDS$ in a two-dimensional array $LCDS[0..m, 1..n]$. We do not need a full array for $Above$: for a fixed i, the values $Above(i, j)$ for $j = 0, 1, ..., n$ depend only on the previous one and on row $i-1$ of $LCDS$, so one variable a holds the current value $Above(i, j-1)$.

Every entry $LCDS[i, j]$ depends only on $LCDS[i-1, j]$ and on $Above(i, j-1)$, and $Above(i, j-1)$ depends only on row $i-1$ at columns $1, ..., j-1$. So we fill the rows in increasing order of i, and inside each row we fill the columns in increasing order of j.

```
LongestCommonDecreasing(A[1..m], B[1..n]):
  for j ← 1 to n: LCDS[0, j] ← 0
  for i ← 1 to m
    a ← 0                                   ⟨⟨a = Above(i, 0)⟩⟩
    for j ← 1 to n
      if A[i] = B[j]
        LCDS[i, j] ← max{LCDS[i − 1, j], 1 + a}
      else
        LCDS[i, j] ← LCDS[i − 1, j]
      if B[j] > A[i]
        a ← max{a, LCDS[i − 1, j]}          ⟨⟨now a = Above(i, j)⟩⟩
  best ← 0
  for j ← 1 to n: best ← max{best, LCDS[m, j]}
  return best
```

Inside row i, the algorithm uses a to compute $LCDS[i, j]$ **before** it adds column j to a, so a is $Above(i, j-1)$ at that time, as the recurrence needs.

Running time

The array has $(m+1)n$ entries. Each entry, and each update of a, takes $O(1)$ time. The final loop takes $O(n)$ time. So the algorithm runs in

$$\boxed{O(mn) \text{ time.}}$$

Sources and collaborators: None.

LLM use: I used Claude (Anthropic) as a tutor for this problem: it helped me understand the problem and guided me, through explanation and questions, toward a correct approach. I wrote and typeset this entire submission myself.

(b) Longest common palindromic subsequence

The empty sequence is a palindrome, so the answer is always at least 0.

Recursive function

For $1 \le i \le m+1$, $0 \le j \le m$, $1 \le k \le n+1$ and $0 \le l \le n$, define:

$LCPS(i,j,k,l)$ = the length of the longest common subsequence of the substrings $A[i..j]$ and $B[k..l]$ that is a palindrome.

A substring with a start index larger than its end index is empty. We need $LCPS(1, m, 1, n)$.

Recurrence

If $i > j$ or $k > l$, then $LCPS(i,j,k,l) = 0$. Otherwise, $LCPS(i,j,k,l)$ is the maximum of the following options:

$$LCPS(i,j,k,l) = \max \begin{cases} LCPS(i+1,j,k,l) & (\text{skip } A[i]) \\ LCPS(i,j-1,k,l) & (\text{skip } A[j]) \\ LCPS(i,j,k+1,l) & (\text{skip } B[k]) \\ LCPS(i,j,k,l-1) & (\text{skip } B[l]) \\ 1 & \text{if } A[i] = B[k] \\ 2 + LCPS(i+1,j-1,k+1,l-1) & \text{if } i < j,\ \ k < l, \text{ and } A[i] = A[j] = B[k] = B[l] \end{cases}$$

Base case: If $i > j$ or $k > l$, one of the two substrings is empty. The only common subsequence is the empty sequence, so the answer is 0.

Every option is possible: The four "skip" options use a common palindromic subsequence of shorter substrings, which is also a common palindromic subsequence of $A[i..j]$ and $B[k..l]$. If $A[i] = B[k]$, the one-element sequence $A[i]$ is a common palindromic subsequence. If $A[i] = A[j] = B[k] = B[l] = c$ with $i < j$ and $k < l$, let Q be a common palindromic subsequence of $A[i+1..j-1]$ and $B[k+1..l-1]$. Then cQc is a palindrome, and it is a subsequence of $A[i..j]$ and of $B[k..l]$. Its length is $2 + |Q|$.

The best option is at least the answer: Let P be a longest common palindromic subsequence of $A[i..j]$ and $B[k..l]$, with $i \le j$ and $k \le l$. Fix one embedding of P into $A[i..j]$ and one embedding into $B[k..l]$. We use the following cases:

- ***P* is empty:** Every option is at least 0.
- **The embedding into *A* does not use *A*[*i*] or does not use *A*[*j*]:** Then P is a common palindromic subsequence of $A[i+1..j]$ or of $A[i..j-1]$, together with $B[k..l]$. A "skip $A[i]$" or "skip $A[j]$" option is at least $|P|$.

- **The embedding into B does not use $B[k]$ or does not use $B[l]$:** By the same argument, a "skip $B[k]$" or "skip $B[l]$" option is at least $|P|$.
- **Both embeddings use all four endpoints, and $|P| = 1$:** The only element of P is matched to $A[i] = A[j]$ and to $B[k] = B[l]$, so $A[i] = B[k]$. The option 1 equals $|P|$.
- **Both embeddings use all four endpoints, and $|P| \geq 2$:** The first element of P is matched to $A[i]$ and $B[k]$, and the last element of P is matched to $A[j]$ and $B[l]$. The first and last elements of P are different elements, so $i < j$ and $k < l$. Because P is a palindrome, its first and last elements are equal, so $A[i] = A[j] = B[k] = B[l]$. Remove both of them and call the rest Q. Then Q is a palindrome, and it is a common subsequence of $A[i+1..j-1]$ and $B[k+1..l-1]$. So $|P| = 2 + |Q| \leq 2 + LCPS(i+1, j-1, k+1, l-1)$.

So the maximum of the options is exactly $LCPS(i, j, k, l)$.

Memoization and evaluation order

We store $LCPS$ in a four-dimensional array $LCPS[1..m+1, 0..m, 1..n+1, 0..n]$. Every entry with $i > j$ or $k > l$ is 0.

Each entry depends only on entries with a larger i, or the same i and a smaller j, or the same (i, j) and a larger k, or the same (i, j, k) and a smaller l. So we use four nested loops, from the outside in: i decreases from m to 1, j increases from i to m, k decreases from n to 1, and l increases from k to n.

```
LongestCommonPalindrome(A[1..m], B[1..n]):
  set every entry of LCPS to 0      ⟨⟨covers i > j and k > l⟩⟩
  for i ← m down to 1
    for j ← i to m
      for k ← n down to 1
        for l ← k to n
          v ← max{LCPS[i+1, j, k, l], LCPS[i, j−1, k, l]}
          v ← max{v, LCPS[i, j, k+1, l], LCPS[i, j, k, l−1]}
          if A[i] = B[k]: v ← max{v, 1}
          if i < j and k < l and A[i] = A[j] = B[k] = B[l]
            v ← max{v, 2 + LCPS[i+1, j−1, k+1, l−1]}
          LCPS[i, j, k, l] ← v
  return LCPS[1, m, 1, n]
```

Running time

The array has $(m+1)^2(n+1)^2 = O(m^2n^2)$ entries. Each entry is computed in $O(1)$ time from at most five other entries. So the algorithm runs in

$O(m^2n^2)$ time.

Sources and collaborators: None.

LLM use: I used Claude (Anthropic) as a tutor for this problem: it helped me understand the problem and guided me, through explanation and questions, toward a correct approach. I wrote and typeset this entire submission myself.